

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 1**  
**MODUL 18**  
**“MESIN ABSTRAK”**



**DISUSUN OLEH:**

**Rasya Putra Wibowo**

**109082500132**

**S1 IF-13-02**

**DOSEN:**

**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2025/2026**

## **DASAR TEORI**

Mesin abstrak merupakan suatu model konseptual dalam ilmu komputer yang digunakan untuk menggambarkan cara suatu sistem mengolah data tanpa bergantung pada bentuk atau mekanisme perangkat keras yang digunakan. Pendekatan ini memungkinkan pengembang berfokus pada logika, struktur data, serta algoritma yang digunakan dalam menyelesaikan suatu permasalahan, sehingga proses perancangan program menjadi lebih sederhana dan sistematis.

Di dalam sebuah mesin abstrak terdapat sekumpulan tipe data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut. Setiap operasi dirancang sesuai dengan kebutuhan sistem yang akan dibangun, sehingga dapat merepresentasikan proses yang terjadi di dunia nyata ke dalam bentuk yang lebih mudah dipahami dan diimplementasikan. Dengan konsep ini, pengembangan perangkat lunak menjadi lebih terstruktur karena setiap fungsi memiliki peran yang jelas.

Salah satu penerapan mesin abstrak dapat ditemukan pada permainan domino. Pada kasus ini, objek utama yang digunakan adalah kartu domino yang memiliki dua sisi dengan jumlah titik (pip) tertentu. Mesin abstrak domino menyediakan berbagai operasi dasar, seperti mengocok kartu, mengambil kartu dari tumpukan, menampilkan nilai atau gambar kartu, membandingkan kartu, hingga menentukan kartu yang dapat dimainkan sesuai aturan permainan. Operasi-operasi tersebut menggambarkan alur permainan tanpa harus memperhatikan bagaimana proses tersebut dijalankan oleh komputer.

Penerapan konsep mesin abstrak memberikan banyak keuntungan dalam proses pengembangan perangkat lunak. Selain membuat program lebih mudah dipahami, pendekatan ini juga meningkatkan modularitas sehingga setiap bagian program dapat dikembangkan atau diperbaiki secara terpisah tanpa memengaruhi keseluruhan sistem. Dengan demikian, mesin abstrak menjadi salah satu konsep fundamental dalam rekayasa perangkat lunak karena membantu menghasilkan program yang lebih terorganisir, mudah dipelihara, serta lebih fleksibel untuk dikembangkan di masa mendatang.

## SOAL LATIHAN

### 1. Mesin Abstrak Dasar Domino

#### Source Code:

```
//Rasya Putra Wibowo_109082500132
package main

import "fmt"

type Domino struct {
    sisiA int
    sisiB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu      [28]Domino
    jumlahSisa int
}

func kocokKartu(d *Dominoes) {
    var i int
    var j int
    var indeks int
    var seed int

    indeks = 0
    for i = 0; i <= 6; i++ {
        for j = i; j <= 6; j++ {
            d.kartu[indeks].sisiA = i
            d.kartu[indeks].sisiB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.jumlahSisa = 28

    fmt.Println("Masukkan angka seed (bilangan bulat) untuk mengacak kartu:")
    fmt.Scan(&seed)

    for i = 27; i >= 1; i-- {
        seed = (seed*1103515245 + 12345) % 2147483648
        if seed < 0 {
            seed = seed * -1
        }
        j = seed % (i + 1)
```

```

        var temp Domino
        temp = d.kartu[i]
        d.kartu[i] = d.kartu[j]
        d.kartu[j] = temp
    }
}

func ambilKartu(d *Dominoes) Domino {
    var hasil Domino
    if d.jumlahSisa <= 0 {
        hasil.sisiA = -1
        hasil.sisiB = -1
        hasil.nilai = -1
        hasil.balak = false
        return hasil
    }
    hasil = d.kartu[d.jumlahSisa-1]
    d.jumlahSisa = d.jumlahSisa - 1
    return hasil
}

func gambarKartu(kartu Domino, suit int) int {
    if kartu.sisiA == suit {
        return kartu.sisiB
    }
    if kartu.sisiB == suit {
        return kartu.sisiA
    }
    return -1
}

func nilaiKartu(kartu Domino) int {
    return kartu.nilai
}

func main() {
    var dominoes Dominoes
    var i int
    var jumlahAmbil int
    var kartuDiambil [28]Domino
    var suitInput int

    fmt.Println("=== Soal 1: Implementasi mesin abstrak kartu domino ===")

    kocokKartu(&dominoes)

    fmt.Println("\n[1b] Urutan 28 kartu setelah kocokKartu:")
    for i = 0; i < 28; i++ {
        fmt.Printf("(%d,%d) ", dominoes.kartu[i].sisiA,
            dominoes.kartu[i].sisiB)
    }
    fmt.Println()
    fmt.Println("Jumlah kartu sisa:", dominoes.jumlahSisa)
}

```

```

        fmt.Println("\n[1c] Masukkan jumlah kartu yang ingin
diambil dari tumpukan:")
        fmt.Scan(&jumlahAmbil)

        for i = 0; i < jumlahAmbil; i++ {
            kartuDiambil[i] = ambilKartu(&dominoes)
            fmt.Printf("Kartu ke-%d diambil: (%d,%d), nilai=%d,
balak=%t\n",
                    i+1, kartuDiambil[i].sisiA,
kartuDiambil[i].sisiB, kartuDiambil[i].nilai,
kartuDiambil[i].balak)
        }
        fmt.Println("Jumlah kartu sisa setelah pengambilan:",
dominoes.jumlahSisa)

        fmt.Println("\n[1d] Masukkan nilai suit (gambar) yang
ingin dicari pada kartu pertama yang diambil:")
        fmt.Scan(&suitInput)
        fmt.Printf("gambarKartu pada (%d,%d) dengan suit=%d
menghasilkan: %d\n",
                    kartuDiambil[0].sisiA, kartuDiambil[0].sisiB,
suitInput, gambarKartu(kartuDiambil[0], suitInput))

        fmt.Println("\n[1e] Nilai dari setiap kartu yang
diambil:")
        for i = 0; i < jumlahAmbil; i++ {
            fmt.Printf("nilaiKartu(%d,%d) = %d\n",
kartuDiambil[i].sisiA, kartuDiambil[i].sisiB,
nilaiKartu(kartuDiambil[i]))
        }

        fmt.Println("Rasya Putra Wibowo_109082500132")
    }
}

```

**Output:**

```
111     fmt.Println("Jumlah kartu sisa setelah pengambilan:", dominoes.jumlahSisa)
112
113     fmt.Println("\n[1d] Masukkan nilai suit (gambar) yang ingin dicari pada kartu pertama yang dian
114     fmt.Scan(&suitInput)
115     fmt.Printf("gambarKartu pada (%d,%d) dengan suit=%d menghasilkan: %d\n",
116         kartuDiambil[0].sisiA, kartuDiambil[0].sisiB, suitInput, gambarKartu(kartuDiambil[0], suitI
117
118     fmt.Println("\n[1e] Nilai dari setiap kartu yang diambil:")
119     for i = 0; i < jumlahAmbil; i++ {
120         fmt.Printf("nilaiKartu(%d,%d) = %d\n", kartuDiambil[i].sisiA, kartuDiambil[i].sisiB, nilaiK
121     }
122
123     fmt.Println("Rasya Putra Wibowo_109082500132")
124 }
125
```

PROBLEMS 22 OUTPUT TERMINAL PORTS

> TERMINAL powershell

```
PS C:\Users\ASUS\Downloads\Modul18_Rasya Putra Wibowo_109082500132_S1IF-13-02> go run dasarDomino.go
Kartu ke-1 diambil: (1,6), nilai=7, balak=false
Kartu ke-2 diambil: (3,6), nilai=9, balak=false
Kartu ke-3 diambil: (3,5), nilai=8, balak=false
Jumlah kartu sisa setelah pengambilan: 25

[1d] Masukkan nilai suit (gambar) yang ingin dicari pada kartu pertama yang diambil:
1
gambarKartu pada (1,6) dengan suit=1 menghasilkan: 6

[1e] Nilai dari setiap kartu yang diambil:
nilaiKartu(1,6) = 7
nilaiKartu(3,6) = 9
nilaiKartu(3,5) = 8
Rasya Putra Wibowo_109082500132
```

### Deskripsi Program:

Program ini bikin dulu struct Domino buat nyimpen data satu kartu (dua sisi gambar, nilai total, dan status balak), lalu struct Dominoes buat nyimpen 28 kartu sekaligus dalam bentuk array plus penghitung sisa kartu. Pas program jalan, fungsi kocokKartu otomatis bikin 28 kombinasi kartu domino dari (0,0) sampai (6,6), terus minta user masukin angka **seed**, angka ini dipakai sebagai "kunci" buat ngacak urutan kartunya pakai rumus acak sendiri (soalnya gak boleh pakai math/rand). Habis itu user diminta masukin **berapa kartu yang mau diambil**, dan setiap kartu yang diambil otomatis kepanggil pakai ambilKartu yang ngambil dari urutan paling atas/terakhir dari tumpukan. Terakhir user masukin **nilai suit** buat dicoba dicari pasangannya di kartu pertama yang udah diambil tadi, pakai gambarKartu, dan semua kartu yang diambil dihitung nilainya pakai nilaiKartu.

## 2. Realisasi Mesin Abstrak Domino

### Source Code:

```
//Rasya Putra Wibowo_109082500132
package main

import "fmt"

type Domino struct {
    sisiA int
    sisiB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu      [28]Domino
    jumlahSisa int
}

func kocokKartu(d *Dominoes) {
    var i int
    var j int
    var indeks int
    var seed int

    indeks = 0
    for i = 0; i <= 6; i++ {
        for j = i; j <= 6; j++ {
            d.kartu[indeks].sisiA = i
            d.kartu[indeks].sisiB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.jumlahSisa = 28

    fmt.Println("Masukkan angka seed (bilangan bulat) untuk
mengacak kartu:")
    fmt.Scan(&seed)

    for i = 27; i >= 1; i-- {
        seed = (seed*1103515245 + 12345) % 2147483648
        if seed < 0 {
            seed = seed * -1
        }
        j = seed % (i + 1)
```

```

        var temp Domino
        temp = d.kartu[i]
        d.kartu[i] = d.kartu[j]
        d.kartu[j] = temp
    }
}

func ambilKartu(d *Dominoes) Domino {
    var hasil Domino
    if d.jumlahSisa <= 0 {
        hasil.sisiA = -1
        hasil.sisiB = -1
        hasil.nilai = -1
        hasil.balak = false
        return hasil
    }
    hasil = d.kartu[d.jumlahSisa-1]
    d.jumlahSisa = d.jumlahSisa - 1
    return hasil
}

func galiKartu(d *Dominoes, kartuAcuan Domino) Domino {
    var kartuDigali Domino
    var ditemukan bool

    ditemukan = false
    for d.jumlahSisa > 0 && ditemukan == false {
        kartuDigali = ambilKartu(d)
        if kartuDigali.sisiA == kartuAcuan.sisiA ||
kartuDigali.sisiA == kartuAcuan.sisiB ||
        kartuDigali.sisiB == kartuAcuan.sisiA ||
kartuDigali.sisiB == kartuAcuan.sisiB {
            ditemukan = true
        }
    }

    if ditemukan == false {
        kartuDigali.sisiA = -1
        kartuDigali.sisiB = -1
        kartuDigali.nilai = -1
        kartuDigali.balak = false
    }
    return kartuDigali
}

func sepasangKartu(kartu1 Domino, kartu2 Domino) bool {
    var total int
    total = kartu1.nilai + kartu2.nilai
    if total == 12 {
        return true
    }
    return false
}

func main() {
    var dominoes Dominoes

```



```

var kartuAcuan Domino
var kartuGalian Domino
var sisiAInput int
var sisiBInput int

fmt.Println("=== Soal 2: Realisasi aksi galiKartu &
sepasangKartu ===")

kocokKartu(&dominoes)

fmt.Println("\n[2a] Masukkan sisiA kartu acuan (0-6):")
fmt.Scan(&sisiAInput)
fmt.Println("Masukkan sisiB kartu acuan (0-6):")
fmt.Scan(&sisiBInput)

kartuAcuan.sisiA = sisiAInput
kartuAcuan.sisiB = sisiBInput
kartuAcuan.nilai = sisiAInput + sisiBInput
if sisiAInput == sisiBInput {
    kartuAcuan.balak = true
} else {
    kartuAcuan.balak = false
}

fmt.Printf("Kartu acuan: (%d,%d)\n", kartuAcuan.sisiA,
kartuAcuan.sisiB)
fmt.Println("Sisa kartu di tumpukan sebelum menggali:",
dominoes.jumlahSisa)

kartuGalian = galiKartu(&dominoes, kartuAcuan)
fmt.Printf("Kartu hasil galiKartu: (%d,%d)\n",
kartuGalian.sisiA, kartuGalian.sisiB)
fmt.Println("Sisa kartu di tumpukan setelah menggali:",
dominoes.jumlahSisa)

fmt.Println("\n[2b] Uji sepasangKartu pada kartu acuan
dan kartu hasil galian:")
fmt.Printf("sepasangKartu((%d,%d), (%d,%d)) = %t
(nilai1=%d + nilai2=%d = %d)\n",
    kartuAcuan.sisiA, kartuAcuan.sisiB,
kartuGalian.sisiA, kartuGalian.sisiB,
    sepasangKartu(kartuAcuan, kartuGalian),
    kartuAcuan.nilai, kartuGalian.nilai,
kartuAcuan.nilai+kartuGalian.nilai)

fmt.Println("Rasya Putra Wibowo_109082500132")
}

```

**Output:**

```
1 //Rasya Putra Wibowo_109082500132
2 package main
3
4 import "fmt"
5
6 type Domino struct {
7     sisiA int
8     sisiB int
9     nilai int
10    balak bool
11 }
12
13 type Dominoes struct {
14     kartu [28]Domino
15     jumlahSisa int
16 }
17
18 func KocokKartu(d *Dominoes) {
19     // ...
20 }
21
22 func SepasangKartu(k1, k2 Domino) bool {
23     // ...
24 }
25
26 func GaliKartu(d *Dominoes) Domino {
27     // ...
28 }
29
30 func main() {
31     // ...
32 }
```

PROBLEMS 22 OUTPUT TERMINAL PORTS

TERMINAL powershell

PS C:\Users\ASUS\Downloads\Modul18\_Rasya Putra Wibowo\_109082500132\_S1IF-13-02> go run realisasiDomino.go

7

[2a] Masukkan sisiA kartu acuan (0-6):  
27  
Masukkan sisiB kartu acuan (0-6):  
1  
Kartu acuan: (27,1)  
Sisa kartu di tumpukan sebelum menggali: 28  
Kartu hasil galiKartu: (1,6)  
Sisa kartu di tumpukan setelah menggali: 27

[2b] Uji sepasangKartu pada kartu acuan dan kartu hasil galian:  
sepasangKartu((27,1), (1,6)) = false (nilai1=28 + nilai2=7 = 35)  
Rasya Putra Wibowo 109082500132

### Deskripsi Program:

Lanjutan dari soal 1, di sini ditambah dua fungsi baru. galiKartu itu simulasi nyari kartu di tumpukan yang gambarnya nyambung sama kartu acuan yang dimasukan user — program bakal ambil kartu satu-satu dari tumpukan sampai nemu yang salah satu sisinya sama kayak kartu acuan. sepasangKartu cuma ngecek dua kartu, kalau jumlah nilainya pas 12 maka hasilnya true, kalau bukan false.

### 3. Gapleh

#### Source Code:

```
//Rasya Putra Wibowo_109082500132
package main

import "fmt"

type Domino struct {
```

```

        sisiA int
        sisiB int
        nilai int
        balak bool
    }

type Dominoes struct {
    kartu      [28]Domino
    jumlahSisa int
}

func kocokKartu(d *Dominoes) {
    var i int
    var j int
    var indeks int
    var seed int

    indeks = 0
    for i = 0; i <= 6; i++ {
        for j = i; j <= 6; j++ {
            d.kartu[indeks].sisiA = i
            d.kartu[indeks].sisiB = j
            d.kartu[indeks].nilai = i + j
            if i == j {
                d.kartu[indeks].balak = true
            } else {
                d.kartu[indeks].balak = false
            }
            indeks = indeks + 1
        }
    }
    d.jumlahSisa = 28

    fmt.Println("Masukkan angka seed (bilangan bulat) untuk
mengacak kartu:")
    fmt.Scan(&seed)

    for i = 27; i >= 1; i-- {
        seed = (seed*1103515245 + 12345) % 2147483648
        if seed < 0 {
            seed = seed * -1
        }
        j = seed % (i + 1)

        var temp Domino
        temp = d.kartu[i]
        d.kartu[i] = d.kartu[j]
        d.kartu[j] = temp
    }
}

func ambilKartu(d *Dominoes) Domino {
    var hasil Domino
    if d.jumlahSisa <= 0 {
        hasil.sisiA = -1
        hasil.sisiB = -1
    }
}

```

```

        hasil.nilai = -1
        hasil.balak = false
        return hasil
    }
    hasil = d.kartu[d.jumlahSisa-1]
    d.jumlahSisa = d.jumlahSisa - 1
    return hasil
}

func cocokUjung(ujung int, kartuBaru Domino) (bool, int) {
    if kartuBaru.sisiA == ujung {
        return true, kartuBaru.sisiB
    }
    if kartuBaru.sisiB == ujung {
        return true, kartuBaru.sisiA
    }
    return false, ujung
}

func mainGapleh(d *Dominoes) ([28]Domino, int) {
    var rangkaian [28]Domino
    var panjangRangkaian int
    var kartuAwal Domino
    var ujungKanan int
    var kartuLewat [28]Domino
    var jumlahLewat int
    var i int
    var kartuBaru Domino
    var cocok bool
    var ujungBaru int
    var masihAda bool

    jumlahLewat = 0
    panjangRangkaian = 0

    if d.jumlahSisa > 0 {
        kartuAwal = ambilKartu(d)
        rangkaian[0] = kartuAwal
        panjangRangkaian = 1
        ujungKanan = kartuAwal.sisiB
    }

    for d.jumlahSisa > 0 {
        kartuBaru = ambilKartu(d)
        cocok, ujungBaru = cocokUjung(ujungKanan, kartuBaru)
        if cocok {
            rangkaian[panjangRangkaian] = kartuBaru
            panjangRangkaian = panjangRangkaian + 1
            ujungKanan = ujungBaru
        } else {
            kartuLewat[jumlahLewat] = kartuBaru
            jumlahLewat = jumlahLewat + 1
        }
    }

    masihAda = true

```

```

        for masihAda {
            masihAda = false
            for i = 0; i < jumlahLewat; i++ {
                cocok, ujungBaru = cocokUjung(ujungKanan,
kartuLewat[i])
                if cocok {
                    rangkaian[panjangRangkaian] =
kartuLewat[i]
                    panjangRangkaian = panjangRangkaian + 1
                    ujungKanan = ujungBaru
                    kartuLewat[i] = kartuLewat[jumlahLewat-1]
                    jumlahLewat = jumlahLewat - 1
                    masihAda = true
                    break
                }
            }
        }

        return rangkaian, panjangRangkaian
    }

func main() {
    var dominoes Dominoes
    var rangkaian [28]Domino
    var panjang int
    var i int

    fmt.Println("=== Soal 3: Implementasi permainan Gapleh
===")
    kocokKartu(&dominoes)

    fmt.Println("Urutan kartu di tumpukan sebelum bermain:")
    for i = 0; i < 28; i++ {
        fmt.Printf("(%d,%d) ", dominoes.kartu[i].sisiA,
dominoes.kartu[i].sisiB)
    }
    fmt.Println()

    rangkaian, panjang = mainGapleh(&dominoes)

    fmt.Println("\nRangkaian kartu Gapleh yang berhasil
disambungkan:")
    for i = 0; i < panjang; i++ {
        fmt.Printf("(%d,%d) ", rangkaian[i].sisiA,
rangkaian[i].sisiB)
    }
    fmt.Println()
    fmt.Println("Total kartu yang berhasil dirangkai:",
panjang, "dari 28 kartu")

    fmt.Println("Rasya Putra Wibowo_109082500132")
}

```

**Output:**

```
//Rasya Putra Wibowo_109082500132
package main

import "fmt"

type Domino struct {
    sisiA int
    sisiB int
    nilai int
    balak bool
}

type Dominoes struct {
    kartu [28]Domino
    jumlahSisa int
}

func kocakKartu(d *Dominoes) {
```

PROBLEMS 22 OUTPUT TERMINAL PORTS

TERMINAL powershell + -

```
PS C:\Users\ASUS\Downloads\Modul18_Rasya Putra Wibowo_109082500132_S1IF-13-02> go run gapleh.go
=== Soal 3: Implementasi permainan Gapleh ===
Masukkan angka seed (bilangan bulat) untuk mengacak kartu:
11
Urutan kartu di tumpukan sebelum bermain:
(2,6) (0,5) (2,3) (3,5) (0,3) (1,1) (1,3) (1,5) (0,6) (2,4) (0,1) (0,4) (3,4) (3,6) (2,2) (3,3) (0,0) (4,4) (4,5) (6,6)
(5,5) (1,4) (0,2) (1,6) (5,6) (2,5) (4,6) (1,2)

Rangkaian kartu Gapleh yang berhasil disambungkan:
(1,2) (2,5) (5,6) (1,6) (1,4) (4,5) (1,5) (1,3) (0,3) (0,5) (5,5) (3,5) (3,3) (2,3) (0,2) (0,6) (4,6) (2,4) (2,6) (6,6)
(3,6) (3,4) (0,4) (0,1) (1,1)
Total kartu yang berhasil dirangkai: 25 dari 28 kartu
Rasya Putra Wibowo_109082500132
PS C:\Users\ASUS\Downloads\Modul18_Rasya Putra Wibowo_109082500132_S1IF-13-02>
```

### Deskripsi Program:

Program ini nyimulasiin permainan Gapleh, di mana kartu-kartu disambung satu sama lain kalau gambarnya ketemu. Kartu pertama dari tumpukan otomatis jadi awal rangkaian. Habis itu program ngambil kartu satu-satu, tiap kartu dicek pakai cocokUjung: kalau salah satu sisinya sama kayak ujung kanan rangkaian yang lagi kebentuk, kartu itu disambung; kalau gak nyambung, kartu disimpan dulu di daftar "lewat". Setelah semua kartu di tumpukan habis diambil, program coba sekali lagi nyambungin kartu-kartu yang sempat "lewat" tadi, soalnya bisa jadi sekarang ujung rangkaian udah beda dan jadi nyambung.

## 4. Mesin Karakter

### Source Code:

```
//Rasya Putra Wibowo_109082500132
package main

import "fmt"

type MesinKarakter struct {
    teks    string
    posisi  int
}

func start(m *MesinKarakter, teksBaru string) {
    m.teks = teksBaru
    m.posisi = 0
}

func maju(m *MesinKarakter) {
    m.posisi = m.posisi + 1
}

func eop(m *MesinKarakter) bool {
    if m.posisi >= len(m.teks) {
        return true
    }
    if m.teks[m.posisi] == '.' {
        return true
    }
    return false
}

func cc(m *MesinKarakter) byte {
    if m.posisi >= len(m.teks) {
        return '.'
    }
    return m.teks[m.posisi]
}

func hitungKarakter(m *MesinKarakter, teks string) int {
    var jumlah int
    jumlah = 0
    start(m, teks)
    for eop(m) == false {
        jumlah = jumlah + 1
        maju(m)
    }
    return jumlah
}

func hitungHurufA(m *MesinKarakter, teks string) int {
    var jumlahA int
    jumlahA = 0
```

```

        start(m, teks)
        for eop(m) == false {
            if cc(m) == 'A' {
                jumlahA = jumlahA + 1
            }
            maju(m)
        }
        return jumlahA
    }

func frekuensiHurufA(m *MesinKarakter, teks string) int {
    var totalKarakter int
    var totalA int
    totalKarakter = hitungKarakter(m, teks)
    totalA = hitungHurufA(m, teks)
    if totalKarakter == 0 {
        return 0
    }
    return (totalA * 100) / totalKarakter
}

func hitungKataLE(m *MesinKarakter, teks string) int {
    var jumlahLE int
    var karakterSebelumnya byte
    var adaKarakterSebelumnya bool

    jumlahLE = 0
    adaKarakterSebelumnya = false
    start(m, teks)
    for eop(m) == false {
        if adaKarakterSebelumnya {
            if karakterSebelumnya == 'L' && cc(m) == 'E' {
                jumlahLE = jumlahLE + 1
            }
        }
        karakterSebelumnya = cc(m)
        adaKarakterSebelumnya = true
        maju(m)
    }
    return jumlahLE
}

func main() {
    var mesin MesinKarakter
    var teksUji string

    fmt.Println("=== Soal 4: Mesin abstrak karakter ===")
    fmt.Println("Masukkan untaian karakter (diakhiri tanda titik '.'):")
    fmt.Scanln(&teksUji)

    fmt.Println("\n[4b-i] Jumlah karakter yang terbaca (tanpa tanda titik):")
    fmt.Println(hitungKarakter(&mesin, teksUji))

    fmt.Println("\n[4b-ii] Jumlah huruf 'A' yang terbaca:")

```



```

        fmt.Println(hitungHurufA(&mesin, teksUji))

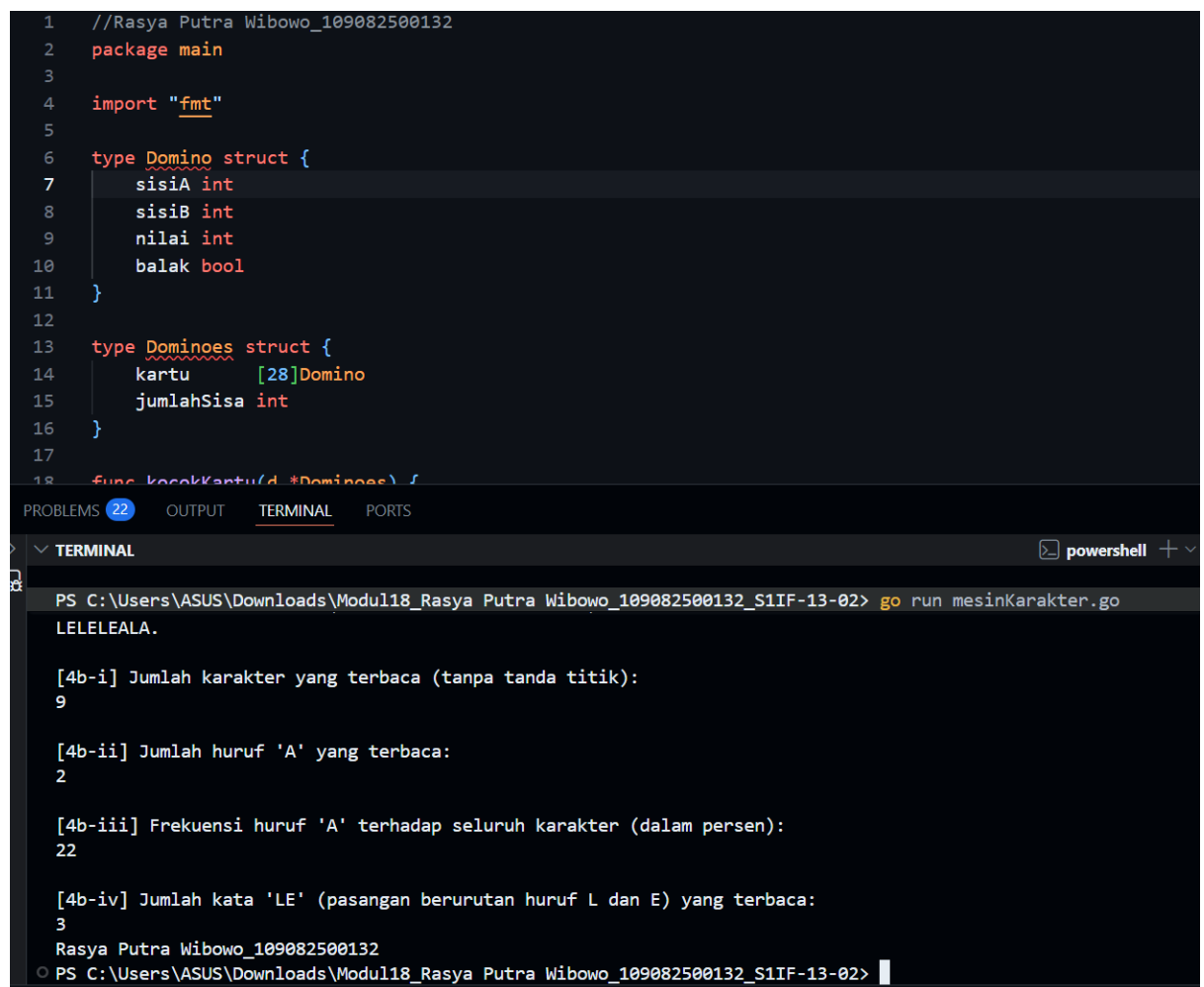
        fmt.Println("\n[4b-iii] Frekuensi huruf 'A' terhadap
seluruh karakter (dalam persen):")
        fmt.Println(frekuensiHurufA(&mesin, teksUji))

        fmt.Println("\n[4b-iv] Jumlah kata 'LE' (pasangan
berurutan huruf L dan E) yang terbaca:")
        fmt.Println(hitungKataLE(&mesin, teksUji))

        fmt.Println("Rasya Putra Wibowo_109082500132")
    }

```

## Output:



The screenshot shows a Go IDE with a source code editor and a terminal window. The source code defines a `Domino` struct with fields `sisia`, `sisib`, `nilai`, and `balak`, and a `Dominoes` struct with fields `kartu` (an array of `Domino`) and `jumlahSisa`. The terminal output shows the execution of the program, displaying the results of the four calculations: the number of characters (9), the frequency of 'A' (2), the frequency of 'A' relative to the total (22), and the number of 'LE' pairs (3). The program ends with the author's name.

```

1 //Rasya Putra Wibowo_109082500132
2 package main
3
4 import "fmt"
5
6 type Domino struct {
7     sisiA int
8     sisiB int
9     nilai int
10    balak bool
11 }
12
13 type Dominoes struct {
14     kartu    [28]Domino
15     jumlahSisa int
16 }
17
18 func kocokKartu(d *Dominoes) {

```

PROBLEMS 22 OUTPUT TERMINAL PORTS

TERMINAL powershell +

```

PS C:\Users\ASUS\Downloads\Modul18_Rasya Putra Wibowo_109082500132_S1IF-13-02> go run mesinKarakter.go
LELELEALA.

[4b-i] Jumlah karakter yang terbaca (tanpa tanda titik):
9

[4b-ii] Jumlah huruf 'A' yang terbaca:
2

[4b-iii] Frekuensi huruf 'A' terhadap seluruh karakter (dalam persen):
22

[4b-iv] Jumlah kata 'LE' (pasangan berurutan huruf L dan E) yang terbaca:
3
Rasya Putra Wibowo_109082500132
PS C:\Users\ASUS\Downloads\Modul18_Rasya Putra Wibowo_109082500132_S1IF-13-02>

```

## Deskripsi Program:

Program ini bikin "mesin" sederhana yang baca karakter satu per satu dari sebuah kalimat sampai nemu tanda titik. Empat operasi dasarnya: start buat nempatin posisi baca di awal kalimat, maju buat geser posisi baca ke karakter selanjutnya, eop

buat ngecek udah sampai titik atau belum, dan cc buat ngambil karakter yang sedang dibaca sekarang. Dari empat operasi dasar itu, dibuat 4 fungsi turunan: ngitung total karakter, ngitung jumlah huruf "A", ngitung persentase huruf "A" dari semua karakter, dan ngitung berapa kali pola "LE" (huruf L diikuti E) muncul berurutan.

## DAFTAR PUSTAKA

- Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley Professional.
- Gabbrielli, M., & Martini, S. (2010). *Programming languages: Principles and paradigms*. Springer.
- Fakultas Informatika, Telkom University. (2026). *Modul 18: Mesin abstrak*. Bahan Ajar Algoritma dan Struktur Data.